



ANKARA YILDIRIM BEYAZIT UNIVERSITY

GRADUATE SCHOOL OF NATURAL AND APPLIED SCIENCES

COMPUTER ENGINEERING

DataGen: Matrix Dataset Generation

By

Mert Altekin 21050111065

Ali Emre Pamuk 21050111021

Faruk Kaplan 21050111026

Supervised by

Fahreddin Şükrü Torun

Date

20.11.2024

ABSTRACT

This report focuses on describing the achievements, inferences, and findings obtained in the development of the project so far. A key accomplishment has been the creation of a simple yet effective graphical user interface (GUI) that facilitates user interaction and enhances functionality. Additionally, matrices have been collected from various public sources, providing a robust dataset for further analysis. The project includes detailed inspection and exploration of matrix features, enabling better understanding and utility. Core functionalities, such as interpolation and extrapolation, have been implemented to allow seamless extensions and predictions using matrix data. Furthermore, matrix operations such as scaling have been integrated, broadening the practical applications of the project. Collectively, these advancements mark significant progress toward achieving the project's objectives.

INTRODUCTION

Background:

This project addresses the challenges of processing large-scale matrices by developing efficient methods for their generation, inspection, and manipulation. These operations are essential in fields such as data analysis, scientific research, and machine learning, where scalable and accurate matrix processing is critical.

Problem Statement:

As matrices grow in size and complexity, traditional methods face limitations in scalability and efficiency. This project aims to integrate dynamic matrix generation and processing functionalities, such as interpolation, extrapolation, and scaling, to enhance computational performance while preserving essential properties.

Objectives:

- **Matrix Processing:** Collect, analyze, and manipulate matrices from public datasets while maintaining critical features.
- **Dynamic Functionality:** Implement interpolation, extrapolation, and scaling techniques to extend the utility of matrices for diverse applications.
- **User Interface:** Provide a simple and effective GUI to improve accessibility and streamline workflows.

Achievements and Progress:

Significant progress has been made, including the development of a user-friendly GUI and the collection and analysis of matrices from public sources. Core functionalities such as interpolation, extrapolation, and scaling have been implemented, expanding the framework's versatility and usability.

Expected Outcomes:

This project is expected to deliver a robust framework for efficient and scalable matrix processing, enabling faster computations, improved accuracy, and minimal manual intervention.

METHODOLOGY / SYSTEM DESIGN

The system is structured into key modules that facilitate matrix collection, inspection, manipulation, and processing, ensuring efficient handling of computational challenges. Significant progress has been made in developing and refining these modules, aligning with the midterm objectives.

Matrix Collection and Inspection Module:

For this module, we gathered matrices from public repositories, such as the SuiteSparse Matrix Collection (e.g., sparse.tamu.edu), and preprocesses them for use. Key features, including the distribution of non-zero elements, row and column statistics, and symmetry, are analyzed to prepare the data for advanced operations. In the coming weeks, tools like ANSYS will be installed and utilized to collect additional matrices with specific properties, further enhancing the dataset's diversity and utility.

Matrix Manipulation Module:

The manipulation module incorporates various functionalities, such as interpolation, extrapolation, and scaling operations. These tools allow users to dynamically adjust matrix properties, extending their usability for different computational scenarios. The implemented graphical user interface (GUI) ensures these functionalities are accessible and user-friendly.

Optimization Module (Future Development):

While still experimental, this module is intended to explore neural network-based techniques to optimize matrix processing further. Plans include evaluating the feasibility of deep learning models for matrix generation and partitioning optimization. Preliminary groundwork has been laid, and future iterations will assess the effectiveness of these models in improving computational efficiency and accuracy.

System Design:

The system employs a modular architecture, enabling seamless integration of advanced processing techniques. Current achievements include matrix collection and manipulation tools that streamline workflows. Future enhancements aim to introduce predictive capabilities, such as determining the optimal number of partitions, to maximize efficiency without compromising precision.

Tools and Technologies:

- **Python** for developing and integrating core functionalities, including matrix manipulation tools.
- **Libraries:** SciPy, NumPy, and custom-built modules for matrix operations.
- **User Interface:** GUI implementation to ensure intuitive interaction with the system.
- **Future Expansion:** TensorFlow/PyTorch for deep learning model development, tools like ANSYS for advanced matrix collection, and parallel computation techniques (e.g., CUDA, MPI, or OpenMP) for optimization tasks.

PROJECT MANAGEMENT

This project will be divided into several phases:

1. **Literature Review** – Research on sparse linear systems and deep learning applications.
2. **Data Collection** – Compilation of matrices from the SuiteSparse Matrix Collection and generated by different applications.
3. **Model Development** – Producing and tuning the various models.
4. **System Implementation** – Integration of the model into code
5. **Testing and Evaluation** – Testing the system's performance across different matrix sizes and properties.

Success Criteria:

- Matrix generation with high accuracy according to input matrix properties
- Improved computational accuracy.

Project Timeline

Task No	Task Name and Aim	Who Will Work on the Task	Timeframe (... Months)	Success Criteria
1	Literature Review and Data Collection: Collect datasets from sparse.tamu.edu and review relevant research on sparse systems and matrix optimization.		1-2 months	Review recent papers; datasets collected.
2	Finding Different matrices		3-4 months	Find different matrices from different resources for applying the model to different fields
3	Model Development: Develop the model to generate matrices while retaining key properties.		4-6 months	Model generates matrices with low amount of error.
4	Final Report Writing and System Testing: Conduct comprehensive system testing, gathering user feedback, and compile the final project report.		7-9 months	Detailed testing report with user feedback.

FUTURE WORKS

Future improvements to the system will aim to expand its applicability beyond sparse systems to encompass broader domains such as machine learning, scientific computing, and predictive modeling. With the foundational GUI and core matrix manipulation functionalities already developed, upcoming efforts will focus on integrating advanced features like automated matrix analysis and predictive tools for determining optimal parameters.

The next steps include incorporating tools like ANSYS to collect matrices with specific properties, further enriching the dataset. Additional testing with larger and more complex matrices under diverse computational conditions will be conducted to evaluate system scalability and robustness.

User feedback will play a vital role in refining the GUI and adding new features to enhance usability. Further optimization may involve exploring distributed computing environments, enabling parallel processing to handle even larger datasets efficiently. The system could also be adapted to work with different types of linear systems beyond sparse matrices, leveraging techniques like interpolation, extrapolation, and scaling in various applications.

Finally, the feasibility of neural network-based methods for matrix generation and optimization will continue to be explored, aiming to develop a hybrid framework that combines deep learning models with traditional computational methods for improved performance.